

Proyecto 3: AWS-S3

Curso: Principios de Sistemas Operativos

Institución: Instituto Tecnológico de Costa Rica

Escuela: Computación

Integrantes: José Fabián Zumbado Ruiz, Cesar Fabricio Herrera Rodriguez

28/06/2026

1. Introducción

El presente documento describe el diseño, implementación y pruebas del Proyecto 3 del curso Principios de Sistemas Operativos del Tecnológico de Costa Rica. El objetivo del proyecto es simular un sistema de almacenamiento de objetos en la nube similar a AWS S3 (Amazon Simple Storage Service), implementado completamente en lenguaje C sobre ambiente Unix.

El sistema consta de dos programas independientes que se comunican mediante sockets TCP: un servidor (**aws-s3_server**) que gestiona los buckets y sus contenidos, y un cliente (**aws-s3**) que permite al usuario realizar operaciones sobre los archivos remotos usando una interfaz de línea de comandos compatible con la sintaxis del AWS CLI.

El proyecto abarca conceptos fundamentales de sistemas operativos como comunicación por sockets, manejo de archivos binarios, gestión de espacio libre en disco, y protocolos de comunicación personalizados.

2. Descripción del Problema

AWS S3 es un servicio de almacenamiento de objetos en la nube que utiliza una arquitectura plana: no existen directorios reales, sino que cada objeto se identifica por una *clave* (key) que puede contener barras diagonales para simular una estructura jerárquica visual. Cada objeto reside dentro de un *bucket*, que es el contenedor de más alto nivel.

Requerimientos del sistema

El sistema debe implementar los siguientes comandos a través del cliente:

Comando	Descripción	Opciones
aws-s3 ls	Lista buckets o contenido de un bucket con prefijos (carpetas simuladas)	—
aws-s3 mb	Crea un nuevo bucket	—
aws-s3 rb	Elimina un bucket; por defecto solo vacío	--force
aws-s3 cp	Copia archivos local ↔ S3 o S3 ↔ S3	--recursive
aws-s3 mv	Mueve un archivo (copia + elimina origen)	--recursive
aws-s3 rm	Elimina objetos de un bucket	--recursive
aws-s3 sync	Sincroniza directorio local con bucket S3	--delete

3. Definición de Estructuras de Datos

3.1 Formato del archivo de bucket

Cada bucket se representa como un único archivo binario en el disco del servidor, ubicado en el directorio `./buckets/` con extensión `.bucket`. Su estructura interna es la siguiente:



3.2 DirEntry — Entrada de directorio

Campo	Descripción
key	Clave del objeto (p.ej. 'fotos/playa.jpg'). Máximo 512 bytes.
offset	Posición en bytes dentro del archivo bucket donde inician los datos.
size	Cantidad de bytes que ocupa este objeto.
is_free	1 = este slot está libre (espacio reutilizable); 0 = activo.

3.3 RequestHeader — Cabecera de solicitud

Cada mensaje cliente → servidor inicia con esta estructura de tamaño fijo:

Campo	Descripción
cmd	Tipo de comando: CMD_LS, CMD_MB, CMD_CP, etc.
src	Ruta fuente (local o s3://)
dst	Ruta destino (local o s3://)
recursive	1 si se paso la opción --recursive
force	1 si se paso la opción --force
delete_flag	1 si se paso la opción --delete
data_len	Cantidad de bytes de payload de archivo que siguen a esta cabecera

3.4 Response Header — Cabecera de respuesta

Campo	Descripción
status	0 = éxito, != 0 = error
message	Mensaje de texto legible para el usuario
data_len	Bytes de payload que siguen (0 si solo es confirmación)

4. Descripción de Componentes Principales

4.1 common/protocol.h — Protocolo compartido

Archivo de cabecera incluido tanto por el cliente como por el servidor. Define todas las estructuras de datos compartidas, las constantes del sistema (puerto, límites de tamaño), y los tipos de comando. Garantiza que ambos extremos de la comunicación interpreten los mensajes de forma idéntica.

4.2 common/protocol.c — Funciones de red

Implementa dos funciones críticas para la transmisión confiable de datos binarios:

```
int send_all(int fd, const void *buf, size_t len);
int recv_all(int fd,          void *buf, size_t len);
```

Ambas funciones iteran hasta enviar/recibir exactamente **len** bytes. Esto es indispensable porque `send()` y `recv()` pueden operar con datos parciales, especialmente en transferencias de archivos grandes. Sin estos wrappers, los datos binarios podrían llegar corrompidos o incompletos.

4.3 server/bucket.c — Gestión de buckets

Módulo central del servidor. Gestiona la creación, lectura y modificación de los archivos de bucket. Sus responsabilidades principales son:

- **bucket_create / bucket_delete**: crea o elimina el archivo `.bucket`.
- **bucket_put**: almacena un objeto. Detecta si ya existe: si el tamaño es igual sobrescribe en el mismo offset; si difiere, marca el espacio antiguo como libre y agrega al final.
- **bucket_get**: lee el objeto del offset indicado en el `DirBlock` y retorna un buffer en memoria.
- **bucket_rm**: marca el `DirEntry` como `is_free=1` sin borrar los datos físicos.
- **bucket_list / bucket_list_all**: construye listados textuales de objetos.
- **bucket_read/write_dirblock**: serializar y deserializa el `DirBlock` al inicio del archivo.

Algoritmo de asignación de espacio libre: First Fit

Cuando se sube un archivo nuevo (o se reemplaza uno de diferente tamaño), `bucket_put` recorre la lista de entradas marcadas como `is_free` y elige la primera cuyo campo **size** sea mayor o igual al tamaño requerido. Si no hay ninguna disponible, el archivo se agrega al final del bucket.

Se eligió el algoritmo **First Fit** por su eficiencia temporal ($O(n)$ en el peor caso) y simplicidad de implementación, adecuada para el alcance de este proyecto académico.

4.4 server/server.c — Servidor TCP

Programa principal del servidor. Crea un socket TCP en el puerto 8080, acepta conexiones secuenciales, lee la `RequestHeader`, y despacha al handler correcto según el campo **cmd**. Cada conexión tiene exactamente una operación.

Flujo principal del servidor:

```
socket() → bind() → listen() → loop:
    accept() ← nueva conexion del cliente
    recv_all(RequestHeader)
```

```

switch(req.cmd):
    CMD_LS      → handle_ls()
    CMD_MB      → handle_mb()
    CMD_RB      → handle_rb()
    CMD_CP      → handle_cp()    ← tambien descarga
    CMD_MV      → handle_mv()    ← cp + rm origen
    CMD_RM      → handle_rm()

    CMD_LIST_KEYS → handle_list_keys()
close(client_fd)

```

4.5 client/client.c — Cliente de linea de comandos

Parsea los argumentos del usuario, construye la RequestHeader apropiada, establece una conexión TCP al servidor, transmite la solicitud (y opcionalmente los bytes del archivo), y muestra la respuesta en consola.

Para las operaciones recursivas (**cp --recursive**, **mv --recursive**, **sync**), el cliente actúa como coordinador: itera sobre los archivos locales con **opendir/readdir** o consulta al servidor con CMD_LIST_KEYS, y luego emite múltiples solicitudes CP individuales, una por cada archivo.

Se ejecutó una suite automatizada de 43 pruebas (**run_tests.sh**) que cubre todos los comandos, casos borde, y la integridad de archivos binarios. Resultado final: **43/43 PASS**.

#	Prueba	Resultado
1	Compilación exitosa con make	PASS
2	Servidor inicia y acepta conexiones	PASS
3	mb: crear bucket nuevo	PASS
4	mb: bucket duplicado retorna error y exit != 0	PASS
5	mb: crear segundo bucket	PASS
6	ls: muestra test-bucket en listado global	PASS
7	ls: muestra other-bucket en listado global	PASS
8	cp: subir archivo de texto	PASS
9	cp: subir archivo binario (PNG con cabecera real)	PASS
11	ls: muestra hello.txt dentro del bucket	PASS
12	ls: muestra photo.png dentro del bucket	PASS
13	ls: muestra big.bin dentro del bucket	PASS
14	ls: filtro por prefijo (solo imgs/)	PASS
15	cp: descargar archivo de texto	PASS
16	cp: contenido descargado idéntico al original (diff)	PASS
17	cp: descargar archivo binario de 64 KB	PASS
18	cp: contenido binario exacto byte por byte	PASS
19	cp: copia S3 -> S3	PASS
20	cp: objeto aparece en bucket destino	PASS
21	cp --recursive: subir directorio con subdirectorios	PASS
22	cp --recursive: docs/report.txt presente en S3	PASS
23	cp --recursive: docs/notes.txt presente en S3	PASS
24	cp --recursive: descarga recursiva	PASS
25	cp --recursive: archivo anidado presente en disco local	PASS
26	mv: mover objeto entre buckets (S3 -> S3)	PASS
27	mv: objeto presente en destino	PASS
28	mv: objeto eliminado del origen	PASS

29	rm: eliminar objeto individual	PASS
30	rm: objeto no aparece en ls posterior	PASS
31	rm --recursive: eliminar prefijo completo	PASS
32	rm --recursive: prefijo no aparece en ls	PASS
33	sync: primera ejecución sube todos los archivos (2)	PASS
34	sync: segunda ejecución omite archivos sin cambios	PASS
35	sync: archivo modificado se re-sube	PASS
36	sync --delete: objeto huérfano eliminado en S3	PASS
37	sync --delete: objeto eliminado no aparece en ls	PASS
38	rb: bucket no vacío retorna error	PASS
39	rb --force: elimina bucket con contenido	PASS
40	rb: bucket no aparece en ls global	PASS
41	Espacio libre: sobreescritura de archivo mas pequeño	PASS
42	Espacio libre: contenido descargado es la nueva versión	PASS
43	Espacio libre: sobreescritura in-place (mismo tamaño)	PASS

Asi mismo, tambien se tiene este script listo para copiar y pegar en consola, además de tener sus resultados también:

```
mkdir -p buckets

# 1. mb - crear buckets
./aws-s3 mb s3://test-bucket
./aws-s3 mb s3://test-bucket          # debe mostrar error
./aws-s3 mb s3://other-bucket

# 2. ls - listar buckets globales
./aws-s3 ls

# 3. cp - subir archivos
echo "Hello world" > hello.txt
printf '\x89PNG\r\n\x1a\n' > photo.png
dd if=/dev/urandom bs=1024 count=64 of=big.bin 2>/dev/null

./aws-s3 cp hello.txt s3://test-bucket/
./aws-s3 cp photo.png s3://test-bucket/imgs/photo.png
./aws-s3 cp big.bin s3://test-bucket/big.bin
```



```
# 4. ls - listar contenido del bucket
./aws-s3 ls s3://test-bucket/
./aws-s3 ls s3://test-bucket/imgs/

# 5. cp - descargar archivos
./aws-s3 cp s3://test-bucket/hello.txt hello_descargado.txt
cat hello_descargado.txt
diff hello.txt hello_descargado.txt && echo "IDENTICOS" || echo "DIFERENTE"

./aws-s3 cp s3://test-bucket/big.bin big_descargado.bin
diff big.bin big_descargado.bin && echo "BINARIO IDENTICO" || echo "DIFERENTE"

# 6. cp - copia S3 a S3
./aws-s3 cp s3://test-bucket/hello.txt s3://other-bucket/copia-hello.txt
./aws-s3 ls s3://other-bucket/

# 7. cp --recursive
mkdir -p src/docs src/imgs
echo "Reporte 2024" > src/docs/report.txt
echo "Notas reunion" > src/docs/notes.txt
echo "imagen" > src/imgs/foto.png

./aws-s3 cp src/ s3://test-bucket/backup/ --recursive
./aws-s3 ls s3://test-bucket/

mkdir -p destino
./aws-s3 cp s3://test-bucket/backup/ destino/ --recursive
find destino -type f

# 8. mv - mover archivo
echo "mover este" > mover.txt
./aws-s3 cp mover.txt s3://test-bucket/mover.txt

./aws-s3 mv s3://test-bucket/mover.txt s3://other-bucket/movido.txt
./aws-s3 ls s3://other-bucket/
./aws-s3 ls s3://test-bucket/

# 9. rm - eliminar objeto
./aws-s3 rm s3://test-bucket/hello.txt
./aws-s3 ls s3://test-bucket/

./aws-s3 rm s3://test-bucket/backup/ --recursive
./aws-s3 ls s3://test-bucket/

# 10. sync
mkdir -p sync-src
echo "Alpha" > sync-src/alpha.txt
echo "Beta" > sync-src/beta.txt
./aws-s3 mb s3://sync-bucket
```

```

./aws-s3 sync sync-src/ s3://sync-bucket/          # sube 2 archivos
./aws-s3 sync sync-src/ s3://sync-bucket/          # no sube nada (sin cambios)

echo "Alpha v2 modificado" > sync-src/alpha.txt
./aws-s3 sync sync-src/ s3://sync-bucket/          # re-sube solo alpha.txt

rm sync-src/beta.txt
./aws-s3 sync sync-src/ s3://sync-bucket/ --delete # elimina beta.txt en S3
./aws-s3 ls s3://sync-bucket/

# 11. rb - eliminar bucket
./aws-s3 rb s3://other-bucket                      # debe dar error (no vacio)
./aws-s3 rb s3://other-bucket --force               # elimina con contenido
./aws-s3 ls

# 12. espacio libre - sobreescritura
./aws-s3 mb s3://free-test
echo "AAAAAAAAAA" > v1.txt
./aws-s3 cp v1.txt s3://free-test/file.txt

echo "BBBBB" > v2.txt
./aws-s3 cp v2.txt s3://free-test/file.txt          # tamano diferente, libera espacio

./aws-s3 cp s3://free-test/file.txt resultado.txt
cat resultado.txt                                    # debe mostrar BBBBB

echo "CCCCC" > v3.txt
./aws-s3 cp v3.txt s3://free-test/file.txt          # mismo tamano, sobreescribe in-place

./aws-s3 cp s3://free-test/file.txt resultado2.txt
cat resultado2.txt                                  # debe mostrar CCCCC

```

Resultados:

```

Built: aws-s3
make_bucket: s3://test-bucket
Error: bucket already exists
make_bucket: s3://other-bucket
OK
s3://test-bucket
s3://Bucket
s3://other-bucket
upload: hello.txt → s3://test-bucket/hello.txt
upload: photo.png → s3://test-bucket/imgs/photo.png
upload: big.bin → s3://test-bucket/big.bin
OK

```

```
hello.txt  (12 bytes)
imgs/photo.png  (8 bytes)
big.bin  (65536 bytes)
OK
    imgs/photo.png  (8 bytes)
OK
Hello world
IDENTICOS
OK
BINARIO IDENTICO
copy: OK
OK
    copia-hello.txt  (12 bytes)
upload: src//imgs/foto.png → s3://test-bucket/backup/imgs/foto.png
upload: src//docs/notes.txt → s3://test-bucket/backup/docs/notes.txt
upload: src//docs/report.txt → s3://test-bucket/backup/docs/report.txt
OK
    hello.txt  (12 bytes)
    imgs/photo.png  (8 bytes)
    big.bin  (65536 bytes)
    backup/imgs/foto.png  (7 bytes)
    backup/docs/notes.txt  (14 bytes)
    backup/docs/report.txt  (13 bytes)
download: s3://test-bucket/backup/imgs/foto.png → destino//imgs/foto.png
OK
download: s3://test-bucket/backup/docs/notes.txt → destino//docs/notes.txt
OK
download: s3://test-bucket/backup/docs/report.txt → destino//docs/report.txt
OK
destino/imgs/foto.png
destino/docs/notes.txt
destino/docs/report.txt
upload: mover.txt → s3://test-bucket/mover.txt
copy: OK
OK
    copia-hello.txt  (12 bytes)
    movido.txt  (11 bytes)
OK
    hello.txt  (12 bytes)
    imgs/photo.png  (8 bytes)
    big.bin  (65536 bytes)
    backup/imgs/foto.png  (7 bytes)
    backup/docs/notes.txt  (14 bytes)
    backup/docs/report.txt  (13 bytes)
delete: s3://test-bucket/hello.txt
```

OK

imgs/photo.png (8 bytes)
big.bin (65536 bytes)
backup/imgs/foto.png (7 bytes)
backup/docs/notes.txt (14 bytes)
backup/docs/report.txt (13 bytes)

delete: removed 3 object(s)

OK

imgs/photo.png (8 bytes)
big.bin (65536 bytes)

make_bucket: s3://sync-bucket

sync upload: sync-src//alpha.txt → s3://sync-bucket/alpha.txt

upload: sync-src//alpha.txt → s3://sync-bucket/alpha.txt

sync upload: sync-src//beta.txt → s3://sync-bucket/beta.txt

upload: sync-src//beta.txt → s3://sync-bucket/beta.txt

sync complete: 2 uploaded, 0 skipped (unchanged), 0 deleted

sync complete: 0 uploaded, 2 skipped (unchanged), 0 deleted

sync upload: sync-src//alpha.txt → s3://sync-bucket/alpha.txt

upload: sync-src//alpha.txt → s3://sync-bucket/alpha.txt

sync complete: 1 uploaded, 1 skipped (unchanged), 0 deleted

sync delete: s3://sync-bucket/beta.txt

delete: s3://sync-bucket/beta.txt

sync complete: 0 uploaded, 1 skipped (unchanged), 1 deleted

OK

alpha.txt (20 bytes)

Error: could not delete bucket (not empty? use --force)

remove_bucket: s3://other-bucket

OK

s3://test-bucket

s3://sync-bucket

s3://Bucket

make_bucket: s3://free-test

upload: v1.txt → s3://free-test/file.txt

upload: v2.txt → s3://free-test/file.txt

OK

BBBBB

upload: v3.txt → s3://free-test/file.txt

OK

CCCCC

fabo@fabo-System-Product-Name:~/Downloads/aws-s3-project\$

4.6 Pruebas de integridad binaria

Se verificó la integridad byte por byte de archivos binarios usando la herramienta **diff** de Unix. Se probaron: una cabecera PNG real (8 bytes), y un archivo de 64 KB generado con **/dev/urandom**. En ambos casos el archivo descargado del servidor fue idéntico al original, confirmando que el protocolo maneja correctamente bytes nulos, bytes de control y secuencias arbitrarias.

4.7 Pruebas del sistema de espacio libre

Se verificó el comportamiento del algoritmo First Fit ante tres escenarios:

- **Archivo de menor tamaño:** el espacio anterior se marca libre, el nuevo se agrega al final, y el contenido recuperado es el nuevo.
- **Archivo del mismo tamaño:** sobrescritura in-place en el mismo offset, sin mover datos.
- **Archivo de mayor tamaño:** el espacio anterior queda marcado como libre para futura reutilización.

Apéndice: Referencia Rápida de Comandos

Todos los comandos disponibles en el cliente **aws-s3**:

```
# Gestionar buckets
./aws-s3 ls                                # listar todos los buckets
./aws-s3 ls s3://mi-bucket/                # listar contenido de un bucket
./aws-s3 ls s3://mi-bucket/fotos/          # listar con prefijo
./aws-s3 mb s3://mi-bucket                 # crear bucket
./aws-s3 rb s3://mi-bucket                 # eliminar bucket vacio
./aws-s3 rb s3://mi-bucket --force          # eliminar bucket con contenido

# Copiar archivos
./aws-s3 cp archivo.txt s3://mi-bucket/    # subir archivo
./aws-s3 cp archivo.txt s3://mi-bucket/docs/x.txt # subir con clave
./aws-s3 cp s3://mi-bucket/x.txt ./        # descargar archivo
./aws-s3 cp s3://a/x.txt s3://b/x.txt      # copiar entre buckets
./aws-s3 cp ./carpeta/ s3://mi-bucket/ --recursive # subir directorio
./aws-s3 cp s3://mi-bucket/ ./destino/ --recursive # descargar directorio

# Mover archivos
./aws-s3 mv archivo.txt s3://mi-bucket/    # subir y eliminar local
./aws-s3 mv s3://a/x.txt s3://b/y.txt      # mover entre buckets
./aws-s3 mv ./dir/ s3://mi-bucket/ --recursive # mover directorio

# Eliminar objetos
./aws-s3 rm s3://mi-bucket/archivo.txt     # eliminar objeto
./aws-s3 rm s3://mi-bucket/fotos/ --recursive # eliminar prefijo completo
```

```
# Sincronizar
./aws-s3 sync ./local/ s3://mi-bucket/          # sincronizar local -> S3
./aws-s3 sync ./local/ s3://mi-bucket/ --delete # sincronizar + eliminar huerfanos
./aws-s3 sync s3://mi-bucket/ ./local/          # sincronizar S3 -> local

# Iniciar el servidor
./aws-s3_server                                # escucha en puerto 8080

# Compilar
make          # compila ambos binarios
make clean    # limpia binarios compilados
bash run_tests.sh  # ejecutar suite de pruebas
```